AI TECHNICAL DEBT PRIORITIZER | SUMMARY 3 OF 3

Design Changes and Their Justification

What changed from the original idea, what would have gone wrong, and why (short version of design document 3)

The original idea had the right direction and most of the right stack. Its gaps were in what it left undefined: what the label is and which period it covers, what the score means, how the fix order is computed, and how anyone would know the model beats a simple rule. The final design makes **39 changes**: **20 fix a flaw**, **17 are improvements** and **2 are scope decisions**.

The three critical fixes

C1 Leakage. Bug-fix counts were a feature and the label came from bug activity over the same history, so the model would copy its input and look excellent while predicting nothing. Now features stop at T and labels start after T.

C11 Random splits. Mixing files of the same project into train and test inflates scores. Now every test is on a later period or an unseen repository.

C38 Untrusted code. A service that clones any URL is open to SSRF, git-hook tricks, resource exhaustion and prompt injection. Now code is parsed, never executed, and every entry point is restricted.

ID	Change	Type	Problem with the original	Why the change is needed or better
Labels, leakage and dataset				
C1	Separate feature and label windows	Fix (Critical)	Same-period bug counts as feature and label: the model copies its input.	Features up to T, labels after T, embargo: a true prediction.
C2	SZZ latent-defect labels	Fix	Every file a fix touches (tests, docs) is marked buggy.	Line-level blame finds where the bug lived; only bugs present at T count.
C3	Defined, graded label	Fix	"Priority" undefined: unreproducible and arguable.	Written formula, grades 0 to 4, sensitivity analysis.
C4	Snapshots and repo criteria	Improve	One snapshot per repo gives about 25 rankings; some repos have no bug data.	About 150 queries and 45,000 rows from repos that track bugs.
C5	Measured bug-fix detection	Fix	Keyword matching is noisy and its error unknown.	Issue links first; precision measured on 200 commits.
Learning objective and scoring				
C6	Learning-to-rank (LambdaMART)	Fix	Classification treats all files equally; users act on the top.	Ranking loss weights mistakes at the top most.
C7	Defined 0 to 100 score	Fix	"92/100" had no unit; not comparable across repos.	Percentile priority plus a calibrated risk probability.
C8	ROI knapsack fix plan	Fix	Sorting by risk picks huge files that use the whole budget.	Most risk removed per hour within the budget (exact DP).
C9	Dependency-ordered plan	Improve	Fixing a module after its dependents means touching them twice.	Cycles collapsed, foundations first, ties by ROI.
C10	Strong baselines	Fix	No proof the model beats churn x complexity or LOC.	Seven baselines including the industry hotspot rule.
Evaluation				
C11	Time and project splits	Fix (Critical)	Random row splits leak project identity and inflate scores.	Temporal split with embargo plus leave-one-project-out.
C12	Ranking, effort-aware metrics	Fix	Accuracy and F1 reward "clean" and favour big files.	NDCG, Precision@10, Recall@20% LOC, P_{opt}, calibration.
C13	Ablation study	Fix	The novelty claim was untested.	Each feature family's contribution is measured.
C14	Significance testing	Improve	With 25 repos, differences may be noise.	Wilcoxon, Cliff's delta, bootstrap intervals.
C15	PROMISE as sanity check	Fix	Different metrics and labels, no history: cannot be merged.	Validates the static extractors without contaminating data.
Features				
C16	Co-change coupling	Improve	Imports miss dynamic, config and cross-language links.	Files that change together are measured as coupled.
C17	Ownership features	Improve	A contributor count cannot tell an owner from scattered edits.	Owner share, minor contributors, entropy, owner active.
C18	Recency-weighted churn	Improve	Five-year-old changes count as much as last month's.	Decayed churn, relative churn, complexity trend.
C19	Graph centrality	Improve	Direct dependents miss indirect reach and cycles.	Transitive dependents, PageRank, betweenness, cycle size.
C20	Static test signals	Fix	Running 150 test suites is infeasible and executes untrusted code.	Test-file and test-import proxies, validated on demo repos.

ID	Change	Type	Problem with the original	Why the change is needed or better
C21	Per-repo normalisation	**Fix**	Metric scales differ hugely between projects.	Percentiles mean the same everywhere and transfer.
C22	SATD feature	**Improve**	Ignores debt developers admit in comments.	NLP classifier adds direct evidence.
C23	Defined duplication method	**Improve**	No method; renamed clones missed.	Token winnowing finds Type-2 clones in linear time.
Explainability and AI layer				
C24	Grouped SHAP sentences	**Fix**	Raw SHAP is unreadable and unstable for correlated features.	Stable family totals in plain language, still exact.
C25	Debt types	**Improve**	A score says how urgent, not what work to do.	Seven types, each with a typical remedy.
C26	Grounded LLM advisor	**Improve**	Generic advice; a naive LLM invents functions.	Schema-validated, AST-checked advice for top files.
C27	What-if simulator	**Improve**	Users cannot see what would change a score.	Toggle actionable features; framed as non-causal.
C28	Feedback loop	**Improve**	No way to adapt to a team or measure agreement.	Pairwise preferences, active learning.
Scope				
C29	File level first	**Scope**	Function tracking across renames is a research problem.	Files now; function metrics used inside files.
C30	Python first, Java next	**Scope**	Thin multi-language support gives unreliable features.	Plug-in interface for the next language.
Engineering and platform				
C31	Background jobs + SSE	**Fix**	Minutes-long analysis times out inside an HTTP request.	Retryable Celery chain with live progress.
C32	PostgreSQL required	**Fix**	No persistence: no caching, trends or feedback.	Source of truth; idempotent analyses.
C33	Incremental analysis	**Improve**	Every run re-parses the whole repository.	Mirrors and blob-hash cache: seconds, not minutes.
C34	Shared core package	**Fix**	Notebook and backend features drift apart.	One package for training and serving, with a parity test.
C35	Versioning and gates	**Improve**	Untraceable scores; pickles can run code.	MLflow, JSON bundles, automatic promotion gates.
C36	GitHub Action + SARIF	**Improve**	A separate dashboard is easy to ignore.	Risk shown in the pull request itself.
C37	Graph library	**Fix**	Recharts cannot draw node-link graphs.	Cytoscape.js for the dependency view.
C38	Security hardening	**Fix (Critical)**	Arbitrary URLs and code: SSRF, hooks, exhaustion, prompt injection.	Allowlist, no code execution, limits, checked LLM output.
C39	Tests, CI/CD, containers	**Improve**	Results and demo depend on one laptop.	Automated checks and a one-command stack.

Kept: the core idea, the self-built dataset, all original inputs (refined), XGBoost, SHAP, Tree-sitter, GitPython, NetworkX, FastAPI, Next.js, Tailwind and Recharts; Random Forest stays as a baseline. **Rejected:** letting an LLM score files directly (not reproducible or calibratable), graph neural networks and end-to-end deep learning (too little data, no readable reasons), a Kaggle or PROMISE dataset as the main data (no history), and Kubernetes (unneeded at this scale).